HOW IT WORKS · IN PLAIN LANGUAGE

How a Migration Works, and Where Everything Is Stored

The full journey of a shop's data — from the old WooCommerce store, through our server, into Shopify — with a real example, the file formats used, what the database keeps, and how long anything is kept.

Version	1.0
Date	6 October 2026
Example used	"Chai & Co" — moving from WooCommerce to Shopify
Status	Agreed design, to be built

Contents

1. The big picture in one page
2. The four places data lives
3. The flow, step by step
4. The three phases: Export, Convert, Import
5. A real example: Chai & Co
6. How files are stored on the server
7. File formats, explained simply
8. What the database keeps (and what it never keeps)
9. The ID map — how orders find their products
10. Big shops: letting Shopify do the heavy work
11. Taking turns — keeping the server and database light
12. How long data is kept, and when it is deleted
13. One customer's journey through the system
14. Words used in this document

1. The big picture in one page

A shop owner wants to move their online shop from **WooCommerce** to **Shopify** — products, customers and past orders. Our app does the moving.

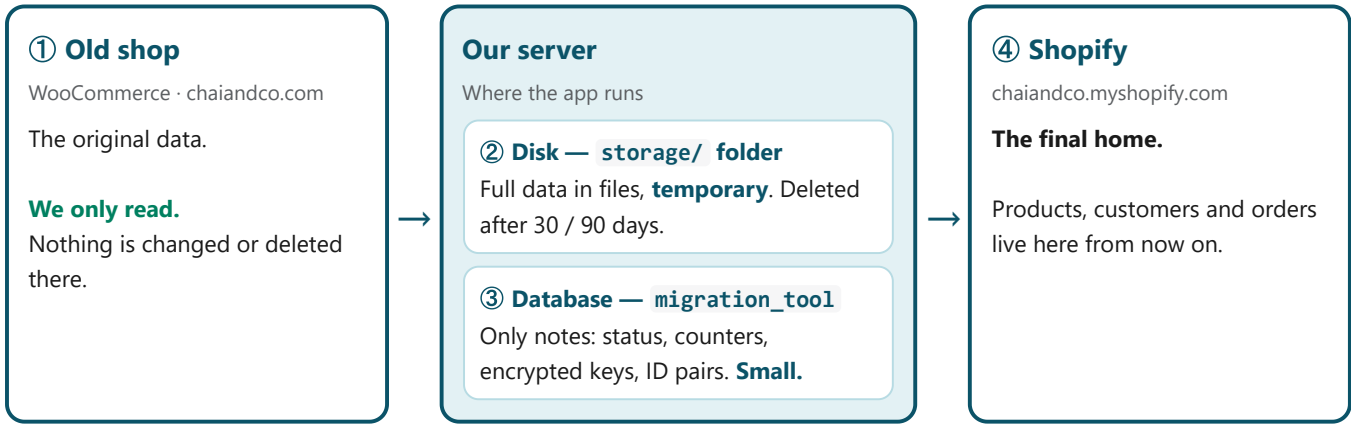
1. Connect Owner gives us read-only keys to the old shop → **2. Choose** Owner ticks what to move → **3. Copy** We copy the data into files on our server → **4. Translate** We rewrite it in Shopify's format → **5. Deliver** We send it into the new Shopify shop → **6. Report** Owner sees what moved and what didn't

The three ideas behind the design

Idea	In simple words
Files carry the data	The actual products, customers and orders are written to **files** on our server for a short time, not into our database. Files are cheap and easy to delete.
The database only keeps notes	The database is like a notebook: "*Migration #7 — products: 2 done, 1 failed.*" It never holds a customer's name, email or address.
Shopify does the heavy lifting	For big shops, we hand Shopify one file and Shopify creates everything on its own servers. Ours just waits.

> **The result:** our database stays small, our server stays fast, and customer data does not pile up on our side — it passes through and is deleted after a set time.

2. The four places data lives



Place	What is there	How long	Who can see it
① Old shop	Original products, customers, orders	Untouched by us	The shop owner
② Server disk	Copies of the data, in files	30 days (customers, orders) / 90 days (rest)	Only our server — never downloadable from the internet
③ Database	Status, counters, encrypted keys, ID pairs	While the app is installed	Only our server
④ Shopify	The migrated shop	Permanent	The shop owner

Important: the files are kept in a private folder called `storage/`. They are **never** put in the `public/` folder — anything in `public/` can be downloaded by anyone on the internet, and these files contain customers' personal details.

3. The flow, step by step

What happens from the moment the shop owner opens the app until the job is finished.

#	What happens	What the owner sees	Behind the scenes
1	Owner installs the app	Setup screen with 3 steps	Shop is saved in the database (its Shopify key encrypted)
2	Owner picks WooCommerce and enters the shop address + API keys	"✓ Connected. Found 3 products, 2 customers, 2 orders"	Keys saved **encrypted**; the counts are noted
3	Owner ticks what to move and sets options	A checklist and a few choices	Choices noted on the migration
4	Owner presses **Begin**	Progress screen, all bars at 0%	Migration joins the **queue**; an empty folder is created for its files
5	The worker's turn comes	"Starting…"	The background worker picks up the job (section 11)
6	**Export**: copy from the old shop	"Reading your old store…"	Data downloaded 50 at a time into `export/` files
7	**Convert**: translate to Shopify format	"Preparing data…"	Each record rewritten into `import/` files; problems go to `errors.csv`
8	**Import**: send into Shopify	Bars fill up: Products, Customers, Orders	Records created in Shopify; old ID ↔ new ID noted in the ID map
9	Redirects created	"Redirects ✓"	Old page addresses point to the new Shopify pages
10	Finished	Results screen + email: moved / failed / why	Status = completed; deletion date set for the files
11	30 / 90 days later	Nothing	Files deleted automatically

The owner can close the browser after step 4.

Everything from step 5 onward happens on our server in the background. The owner can come back any time to watch progress, and gets an email when it's done.

Order of moving

Some data depends on other data — an order needs its products and its customer to exist first. So the steps always run in this order:



4. The three phases: Export, Convert, Import

Every kind of data (products, customers, orders…) goes through the same three phases.

Phase 1 — EXPORT

Old shop → file on our server

What: We ask the old shop for its data, **50 records at a time**, and write each one as a line in a file — exactly as the old shop sent it.

Why in small pages: many shops are on cheap hosting that slows down or crashes if asked for everything at once.

Result: `export/products.jsonl.gz`, `export/customers.jsonl.gz`, `export/orders.jsonl.gz`

If the server stops halfway: the database knows how many pages were done (e.g. 3), so it simply carries on from page 4.

Phase 2 — CONVERT

WooCommerce format → Shopify format

What: We read the export file **one line at a time** and rewrite each record the way Shopify expects it — e.g. WooCommerce's "regular price" and "sale price" become Shopify's "compare-at price" and "price".

Why one line at a time: a file can be huge (hundreds of MB). Reading line by line means the server only ever holds one record in memory.

Problems are caught here. A record Shopify cannot accept (for example a product with 4 options — Shopify allows 3) is not converted. It is written to `errors.csv` with the reason.

Result: `import/products.jsonl` (ready for Shopify) and `errors.csv`

Phase 3 — IMPORT

File → new Shopify shop

Small jobs (a demo, or under ~250 records): we send records to Shopify one by one, slowing down whenever Shopify asks us to.

Big jobs: we upload the whole import file to Shopify and Shopify creates everything itself (see section 10).

After each record is created: we note "*old product 101 is now Shopify product 9001*" in the **ID map** (section 9).

Result: the data is in Shopify; counters updated; any Shopify rejections added to `errors.csv`.

5. A real example: Chai & Co

Riya runs a tea shop, **Chai & Co**, on WooCommerce and is moving it to Shopify. The shop is tiny so every detail fits on the page:

Kind	What the old shop has
3 products	Masala Chai (two sizes: 100g, 250g) · Green Tea · Gift Box (a "grouped" product)
2 customers	Aman (Delhi) · Neha (Pune)
2 orders	#1045 — Aman bought 2 × Masala Chai 250g (completed) · #1046 — Neha bought 1 × Green Tea (processing)

- **Riya connects and presses Begin.** The database gets one migration (#7) and three counters (products, customers, orders). An empty folder is made for the files. Riya closes the laptop.
- **Export.** The worker downloads the 3 products, 2 customers and 2 orders into three files in `export/`.
- **Convert products.** Masala Chai → one Shopify product with two sizes. Green Tea → one product. **Gift Box** is a "grouped" product, which Shopify does not have — so it is written to `errors.csv` instead.
- **Import products.** Shopify creates Masala Chai (*Shopify product 9001*) and Green Tea (*9002*). The ID map now says: *old 101 → 9001*, *old 102 → 9002*, and the same for each size.
- **Customers.** Aman and Neha are created in Shopify (*7001*, *7002*) and get an email inviting them to set a new password. The ID map notes *old 201 → 7001*, *old 202 → 7002*.
- **Orders.** Order #1045 says "customer 201 bought size 112". Shopify doesn't know those numbers — so the worker looks them up in the ID map: *201 → Aman (7001)*, *112 → Masala Chai 250g*. The order is created in Shopify, correctly linked to Aman and the product. **Aman is not emailed** about this old order.
- **Redirects.** `chaiandco.com/product/masala-chai/` now leads to the new Shopify page, so old Google links keep working.
- **Finished.** Riya gets an email: "*6 records migrated, 1 needs attention: Gift Box — grouped product.*" Riya creates the Gift Box by hand in Shopify.
- **30 days later.** The customer and order files are deleted. **90 days later**, the rest of the folder is deleted.
- **Two weeks after launch: "Migrate recent data".** One new order (#1047) arrived on the old shop. The worker re-reads the orders, checks the ID map — #1045 and #1046 are already done — and creates **only #1047**. No duplicates.

6. How files are stored on the server

Every migration gets its own folder. The path is built from three parts:

```
storage / migrations / {platform} / {shop address} / {migration number} /
```

For Chai & Co's migration #7:

```
storage/
└── migrations/
    └── woocommerce/                          ← which platform it came from
        └── chaiandco.myshopify.com/          ← which Shopify shop it is for
            └── 7/                            ← which migration
                ├── export/                   ← RAW copy from the old shop
                │   ├── products.jsonl.gz.enc
                │   ├── customers.jsonl.gz.enc
                │   └── orders.jsonl.gz.enc
                ├── import/                   ← CONVERTED, ready for Shopify
                │   ├── products.jsonl.enc
                │   ├── customers.jsonl.enc
                │   └── orders.jsonl.enc
                ├── errors.csv                ← what failed and why (owner can download)
                └── log.txt                   ← timeline, for our support team
```

Why the path is built this way

Part	Why
platform first	Easy to see at a glance what came from WooCommerce, WordPress, etc.
shop address, not shop name	The address (`chaiandco.myshopify.com`) never changes and is always safe as a folder name. A shop *name* can change and can contain characters folders don't allow.
migration number	A second migration for the same shop gets its own folder and never overwrites the first.
export/ and **import/** kept separately	If something looks wrong in Shopify, support can compare what the old shop sent with what we sent to Shopify.

Rules for the storage folder

- **Private** — outside `public/` , so nothing can be downloaded from a web address.
- **Encrypted** — data files are scrambled with a secret key (the `.enc` ending). Stolen files are unreadable.
- **Not in git** — `storage/` is listed in `.gitignore` , so customer data never ends up in the code repository.
- **Cleaned automatically** — a daily job deletes folders that have passed their keep-until date (section 12).

7. File formats, explained simply

File	Format	Who reads it	Holds
`export/*.jsonl.gz.enc`	JSONL, compressed, encrypted	Our worker	Raw data as the old shop sent it
`import/*.jsonl.enc`	JSONL, encrypted	Our worker, then Shopify	The same data in Shopify's format
`errors.csv`	CSV	The shop owner (Excel / Sheets)	Failed or skipped records and the reason
`log.txt`	Plain text	Our support team	A timeline of the migration

JSONL — "one record per line"

In simple words: a text file where **each line is one complete record** (one product, one customer, one order), written in JSON — a standard way of writing data with labels, like `"name": "Green Tea"`.

Chai & Co's `export/products.jsonl` (shown decrypted):

```
line 1 → {"id":101,"name":"Masala Chai","attributes":[{"name":"Size","options":["100g","250g"]}],"variations":
[{"id":111,"sku":"MC-100","price":"8"},{"id":112,"sku":"MC-250","price":"15"}]}
line 2 → {"id":102,"name":"Green Tea","sku":"GT-1","price":"6"}
line 3 → {"id":103,"name":"Gift Box","type":"grouped","grouped_products":[101,102]}
```

After converting, `import/products.jsonl` — the same products in Shopify's words:

```
line 1 → {"title":"Masala Chai","handle":"masala-chai","productOptions":[{"name":"Size","values":
["100g","250g"]}],"variants":[{"optionValues":["100g"],"sku":"MC-100","price":"8"},{"optionValues":
["250g"],"sku":"MC-250","price":"15"}]}
line 2 → {"title":"Green Tea","handle":"green-tea","variants":[{"sku":"GT-1","price":"6"}]}
        (Gift Box is not here — it went to errors.csv)
```

Why JSONL and not CSV for the data?

CSV is a flat table. One product with 2 sizes and 3 pictures doesn't fit in one row — it has to be split across several files and stitched back together. Slow and easy to get wrong.

JSONL keeps each record whole. A product, all its sizes and all its pictures sit on one line. It can be read one line at a time (low memory), and Shopify accepts JSONL directly for big imports.

CSV — for people

In simple words: a simple table that opens in Excel or Google Sheets. We use it only for the error report, because the shop owner reads it.

```
entity,old_id,name,reason
product,103,Gift Box,"Grouped product — Shopify has no equivalent"
product,118,Linen Shirt,"Has 4 options; Shopify allows 3"
```

The error report holds IDs, product names and reasons only — never customers' emails or addresses.

What ".gz" and ".enc" mean

Ending	Means	Why
`.gz`	**Compressed** (zipped)	Files become 5–10× smaller on disk
`.enc`	**Encrypted** (scrambled with a secret key)	If anyone ever got the files, they could not read them. The key lives only in the server's settings (`.env`).

log.txt — the diary

```
2026-10-06 14:02:10  Migration 7 started (products, customers, orders)
2026-10-06 14:02:14  Export products: 3 read
2026-10-06 14:02:15  Convert products: 2 ready, 1 skipped (Gift Box — grouped)
2026-10-06 14:02:19  Import products: 2 created
2026-10-06 14:02:31  Import orders: 2 created
2026-10-06 14:02:33  Migration 7 completed
```

8. What the database keeps (and what it never keeps)

The database is named `migration_tool`. Think of it as the app's **notebook**: who the shops are, what each migration is doing, and the counters on the progress bars. **The data itself is never written here.**

✓ The database keeps

- Each shop and its plan
- The old shop's address and its **encrypted** keys
- Each migration's status and choices
- Counters: done / failed / skipped
- The folder path of the files
- ID pairs: *old 101 = Shopify 9001*

✗ The database never keeps

- Customer names, emails, phones, addresses
- Order contents or totals
- Product descriptions, prices, images
- The error list or the log (they are files)
- Anything in plain text that is secret

The tables

Table	What it keeps (simple words)	For Chai & Co
`stores`	Each Shopify shop using the app, and its encrypted Shopify key	1 row
`plans`	The price plans and their limits	Shared by everyone
`user_memberships`	Which plan each shop is on	1 row
`membership_payments`	Shopify billing charges	1 row
`dummy_shops`	Test shops that are not charged real money	—
`faqs`	Help questions and answers	Shared by everyone
`source_connections`	The old shop: address, **encrypted** keys, how many records it has	1 row
`migrations`	One row per migration: status, what was ticked, **where its files are**, which phase it is in, **when the files will be deleted**	1 row
`migration_steps`	The counters behind each progress bar: how many exported, imported, failed; and **where to carry on** if stopped	3 rows
`id_map`	Old ID ↔ Shopify ID pairs — numbers only	11 rows

New columns this design adds

Table	Column	Holds
`migrations`	`storage_path`	The folder, e.g. `storage/migrations/woocommerce/chaiandco.myshopify.com/7`
`migrations`	`phase`	export · convert · import · done
`migrations`	`personal_files_expire_at`	When customer and order files are deleted (finish + 30 days)
`migrations`	`files_expire_at`	When the whole folder is deleted (finish + 90 days)
`migration_steps`	`exported`, `imported`	How many records passed each phase

`migration_steps`	`export_page`, `import_line`	Where to carry on after a pause or restart
`migration_steps`	`bulk_operation_id`	Shopify's job number when Shopify is doing a big import

Tables this design removes

Table	Replaced by
`migration_errors`	The `errors.csv` file. The database keeps only the *number* failed.
`migration_logs`	The `log.txt` file.

> **Size check:** a shop with 50,000 orders adds just **1 migration row, a few counter rows, and about 60,000 tiny ID pairs** to the database — no personal data. That is very light work for MySQL.

9. The ID map — how orders find their products

In simple words: the old shop and Shopify give every record a **different number**. The ID map is a list that pairs them up — "*old Masala Chai (101) is Shopify product 9001*" — like a guest list that says which room each guest moved to.

Chai & Co's ID map at the end of the migration:

Kind	Old shop number	Shopify number	Old web address	New web address
product	101	9001	/product/masala-chai/	/products/masala-chai
size	111	9101		
size	112	9102		
product	102	9002	/product/green-tea/	/products/green-tea
customer	201	7001		
customer	202	7002		
order	301	5001		
order	302	5002		

Its five jobs

Job	How it helps
Linking orders	Order #1045 says "customer 201 bought size 112". The map turns that into "Aman (7001) bought Masala Chai 250g (9102)", so the order arrives in Shopify linked to the right person and product.
No duplicates	Before creating anything, we check the map. Already there? Skip it. Each old record can appear only once.
Safe restarts	If the server stops mid-way and repeats some work, the map stops anything being created twice.
Redirects	The old and new web addresses become the 301 redirects that keep Google links working.
Recent data & demo clean-up	"Migrate recent data" moves only what is not yet in the map. "Delete demo data" finds everything the demo created.

The ID map holds numbers and web addresses only — no names, emails or prices — which is why it is safe and small enough to keep in the database.

10. Big shops: letting Shopify do the heavy work

Sending 50,000 orders to Shopify one at a time would keep our server busy for hours. Shopify offers a better way, called a **bulk operation**:

1. Upload
We upload `import/orders.jsonl` to Shopify once

→

2. Start
We tell Shopify: "create everything in this file"

→

3. Shopify works
Shopify creates all records on its own servers

→

4. We check
Every 30 s: "done yet?" — one tiny request

→

5. Results
We download Shopify's result file

Shopify's result file tells us which line became which Shopify record, or why it failed:

```
line 1 → created  gid://shopify/Order/5001
line 2 → created  gid://shopify/Order/5002
line 7 → ERROR    "Line item variant does not exist"
```

We read it line by line, fill the ID map, and add any failures to `errors.csv`.

Job size	How we import	Our server's work
Demo, or under ~250 records	One by one	Small — the job is small
Anything bigger	Shopify bulk operation	Almost none — Shopify does it

11. Taking turns — keeping the server and database light

The queue

> **In simple words:** migrations wait in line, like customers at a counter. A separate program — the **worker** — serves them in turn.

- **One migration at a time per shop.** A shop's second migration waits until its first is finished.
- **At most 2 migrations at once** across the whole server (a setting we can raise as the server grows).
- **Inside a migration, one step at a time**, in the fixed order: collections → products → customers → orders → …

Why the worker is a separate program

The screens shop owners click on are served by the **web app**. Migrations are done by the **worker**. Because they are separate, a huge migration never makes the screens slow for anyone else.

The rules that keep load low

Rule	What it saves
Read and write files **one line at a time**	Memory — a 2 GB file uses about as much memory as a 2 MB one
Update the database **once per 50 records**, not after every record	About 50× fewer database writes
Fetch from the old shop **50 records per request**, with pauses and retries	Doesn't crash the old shop's hosting
Respect Shopify's speed limit; slow down when asked	No blocked requests
Shopify bulk operations for big jobs	Shopify's servers do the creating, not ours
Compress files	5–10× less disk space
Check free disk space before starting an export	Fails cleanly instead of crashing halfway
Delete old files automatically	The disk never fills up

Picking up where it left off

After every page of 50 records, the database notes exactly where the worker is (`export_page` or `import_line`). If the server restarts, the worker reads that note and carries on from the next page. At most one page is repeated — and the ID map stops anything in it being created twice.

When the app grows beyond one server

A folder on one server's disk can only be seen by that server. When we run more than one server, the same folder layout moves to **cloud file storage** (such as Amazon S3 or Cloudflare R2): cheap, practically unlimited, and able to delete old files by itself. Because all file reading and writing goes through one small part of the code, this is a small change.

12. How long data is kept, and when it is deleted

Customer and order files contain **personal details**, so they are kept for a shorter time than the rest. All files are encrypted for as long as they exist.

What	Kept for	Why that long
Customer and order files	**30 days** after finishing	Enough time for the owner to check results and for support questions. Personal data is kept no longer than needed.
Product, collection and page files	**90 days**	Not personal data; useful if a problem is noticed weeks later
`errors.csv` and `log.txt`	**90 days**	Contain IDs and reasons only — no personal details
Database notes and ID map	While the app is installed	Small, no personal data; needed for "migrate recent data"

Both times are settings on the server (e.g. `RETENTION_DAYS_PERSONAL=30`, `RETENTION_DAYS_OTHER=90`) and can be changed without code changes.

Everything that deletes data

When	What is deleted
Every day, automatically	Any files past their keep-until date
Owner presses **"Delete migration data now"**	That migration's whole folder, immediately
A shopper asks to be forgotten (Shopify sends `customers/redact`)	That shop's customer and order files
Owner uninstalls the app (Shopify sends `shop/redact` ~48 hours later)	**Everything** for that shop — all folders and all database rows

> **Backups:** if the server is backed up, those backups contain the files too. Backups must be kept no longer than 90 days, or the `storage/` folder must be left out of them.

The owner is told on the results screen: "*Migration files are kept for 30 days (customers and orders) and 90 days (everything else), then deleted automatically.*" The same is stated in the app's privacy policy, which Shopify requires.

13. One customer's journey through the system

Following **Aman**, a customer of Chai & Co:

When	Where Aman's details are
Before the migration	Old WooCommerce shop
During the migration	Old shop · our server's encrypted files (`export/customers…` , `import/customers…`)
After the migration	Old shop · **Shopify** · our encrypted files
30 days later	Old shop · **Shopify** — our files are deleted
In our database, at any time	Only `customer 201 = Shopify 7001` . **Never** Aman's name, email, phone or address.

> **In one sentence:** the shop's data passes through our server's private, encrypted `storage/` folder for a short time; our database keeps only notes, counters and ID pairs; and the data's permanent home is Shopify.

14. Words used in this document

Word	Meaning
Migration	One job of moving data from the old shop to Shopify.
Old shop / source	The shop being moved from (e.g. WooCommerce).
API keys	A username-and-password pair the old shop creates so our app can read its data. We ask for read-only keys.
Export	Copying data out of the old shop into files.
Convert	Rewriting the data in the format Shopify expects.
Import	Sending the converted data into Shopify.
Worker	A background program on our server that does the migrations, separate from the screens.
Queue	The waiting line of migrations; the worker takes them in turn.
JSONL	A text file with one complete record per line. Used for the data files.
CSV	A simple table that opens in Excel or Google Sheets. Used for the error report.
Compressed (.gz)	Zipped to take less space.
Encrypted (.enc)	Scrambled with a secret key so nobody without the key can read it.
Database	The app's notebook (`migration_tool`) — status, counters, ID pairs.
ID map	The list pairing each old record's number with its new Shopify number.
Bulk operation	Shopify's way of importing a whole file at once on its own servers.
Redirect (301)	An instruction that sends visitors of an old web address to the new one.
Retention	How long we keep files before deleting them.
redact	Shopify's word for "delete this person's / this shop's data".